

Algoritmos, complejidad y estrategias de resolución

TC-CH03 – Técnicas Computacionales en Biología

Edición 2 · 2026-09-03

Pregunta del tema. Un asistente automático escribe en segundos un procedimiento que compara cada lectura de secuenciación con cada posición del genoma. ¿Cómo se decide, antes de lanzarlo, si terminará esta tarde o dentro de nueve años?

Producto final. Una auditoría razonada de tres afirmaciones de coste erróneas, con la medición propia del escalado al duplicar la entrada y la comparación entre una recursión desbocada y su versión con tabla.

Mapa del tema

1. La receta, la despensa y la escala biológica	1
2. El coste de un algoritmo y el modelo de computación	3
3. Notación Asintótica: el lenguaje de la eficiencia	5
4. Las cinco grandes familias de diseño algorítmico	8
5. La operación peligrosa: la explosión combinatoria oculta	12
6. Actividad de depuración: localizar fallos en algoritmos ajenos	13
7. Síntesis y diagnóstico de problemas algorítmicos	13
8. Glosario esencial	14
9. Conexión con el curso y lo que viene después	15
10. Fuentes y límites de uso	15

Cómo usar este tema

Este capítulo no enseña a programar mejor: enseña a decidir, antes de ejecutar nada, si un procedimiento va a terminar. Se lee en tres tiempos. Primero se cuenta: cuántas operaciones hace un algoritmo sobre una entrada de tamaño n . Después se abstrae: qué parte de esa cuenta importa cuando n crece, que es de lo que habla la notación asintótica. Y por último se diseña: cinco familias de estrategias, cada una con su condición de aplicabilidad y su forma característica de fallar.

Al terminar sabrá: contar las operaciones elementales de un fragmento y expresar el resultado como una función de n ; distinguir una cota de un caso, que son cosas distintas y se confunden constantemente; predecir cómo cambia el tiempo al duplicar la entrada y comprobarlo midiendo; reconocer cuándo un algoritmo voraz da la respuesta correcta y cuándo no; y auditar una afirmación de coste ajena, venga de un compañero o de un asistente automático.

La ruta **esencial** son el modelo de coste, el conteo en los tres algoritmos, la notación asintótica con sus tres cotas y las cinco familias de diseño. La ruta de **estudio** añade la demostración del término dominante, la vuelta atrás y el anexo. Prediga siempre antes de medir: este capítulo trata precisamente de que la intuición sobre el coste es mala y hay que corregirla con cuentas.

1 La receta, la despensa y la escala biológica

ESENCIAL

En los dos primeros capítulos aprendimos a desenvolvern en la terminal de Linux y a construir procedimientos automáticos con Bash, tuberías y expresiones regulares. Con esas herramientas podemos procesar archivos de datos, filtrar registros biológicos y encadenar programas. Sin embargo, en cuanto pasamos de pequeños ficheros de prueba a conjuntos de datos biomédicos a gran escala, surge de inmediato un obstáculo insalvable si solo aplicamos fuerza bruta: el **coste computacional**.

Consideremos un ejemplo real en genómica: el genoma humano contiene aproximadamente 3×10^9 pares de bases (3000 millones de nucleótidos). Si un secuenciador de última generación nos entrega 100 millones de lecturas cortas (FASTQ) y diseñamos un procedimiento ingenuo que compare cada lectura posición a posición contra todo el genoma desde el principio, el ordenador tendría que realizar del orden de $100 \times 10^6 \times 3 \times 10^9 = 3 \times 10^{17}$

operaciones. Incluso en un ordenador capaz de ejecutar mil millones de operaciones por segundo, que es un ritmo corriente para un procesador de sobremesa, ese cálculo ingenuo tardaría más de **9 años sin interrupción**.

El cálculo ilustra por qué la fuerza bruta es inviable a escala genómica. Las herramientas bioinformáticas reales no comparan cada lectura contra cada posición: construyen o reutilizan índices estructurados (como tablas de dispersión, árboles de sufijos o la transformada de Burrows-Wheeler, que estudiaremos en los siguientes capítulos) que descartan instantáneamente la inmensa mayoría del espacio de búsqueda. La diferencia entre un cálculo inviable que tardaría años y un análisis rápido no reside en comprar un ordenador más caro, sino en la **eficiencia matemática del algoritmo**.

Si una estructura de datos es la **despensa** —la forma en que se organizan y almacenan los ingredientes en la memoria—, un algoritmo es la **receta**: la secuencia ordenada de instrucciones que transforma los datos de entrada en el resultado biológico deseado. Este capítulo se centra en la receta apoyándose en lo que ya sabemos representar en Bash y pseudocódigo (TC-CH01 y TC-CH02). Las estructuras de datos avanzadas se formalizarán en los capítulos TC-CH04 y TC-CH05; aquí nos basta con saber que las entradas y salidas de un algoritmo son datos, independientemente de dónde y cómo se guarden.

Hoy en día, cualquier asistente de inteligencia artificial puede generar código en segundos, pero con frecuencia elige la estrategia equivocada: fuerza bruta donde existía una solución elegante, o un método voraz que parece resolver el problema pero no garantiza la respuesta óptima global. Reconocer el paradigma que tenemos delante, estimar su coste antes de lanzarlo y juzgar si escalará a nuestros datos es la competencia fundamental que proporciona este capítulo.

Un algoritmo es el procedimiento abstracto que resuelve un problema; un programa es su realización concreta en un lenguaje de programación. El mismo algoritmo puede escribirse como programas distintos en Bash o cualquier otro lenguaje de programación.

Un algoritmo es una secuencia finita de pasos bien definidos que, a partir de una entrada, produce una salida en un tiempo finito. «Finita» y «bien definidos» son las palabras importantes: un algoritmo no puede ser ambiguo ni eterno.

1.1 Niveles de representación de un algoritmo

Antes de lanzarse a teclear código en el ordenador, un algoritmo debe diseñarse y evaluarse conceptualmente. Para ello se utilizan cuatro niveles de representación, ordenados de menor a mayor rigor y precisión:

1. **Lenguaje natural:** describe la idea en lenguaje humano ordinario («recorrer la lista y guardar el número mayor»). Es muy intuitivo para comunicar la idea inicial, pero es propenso a ambigüedades e imprecisiones.
2. **Diagrama de flujo:** representación gráfica normalizada que utiliza cajas y flechas para ilustrar decisiones lógicas, bucles y flujos de datos.
3. **Pseudocódigo:** el punto dulce de la ingeniería. Es un lenguaje estructurado en texto, libre de las rigideces sintácticas de un lenguaje de programación concreto, pero con suficiente rigor lógico para analizar el coste y la corrección del algoritmo. Es la forma estándar en la que razonaremos los algoritmos a lo largo de este libro.
4. **Lenguaje de programación:** código ejecutable (Bash, C u otros lenguajes estructurados) que el sistema operativo puede ejecutar directamente.

Los cuatro niveles de representación de un algoritmo, de menor a mayor precisión, son: lenguaje natural, diagrama de flujo, pseudocódigo y lenguaje de programación.

Concepto. La célula ejecuta algoritmos, y conviene verlo antes de seguir. La replicación del ADN separa las dos hebras con la helicasa y copia cada una con la polimerasa: hay una entrada, la molécula original; una salida, dos moléculas idénticas; y una secuencia de pasos definidos que termina. Su equivalente informático es el problema de la copia: dada una cadena $s = (s_1, s_2, \dots, s_n)$, devolver una copia de s , con coste $O(n)$. Escribir un proceso biológico como entrada, salida y pasos es el primer ejercicio de pensamiento algorítmico, y es también la razón de que la biología se deje analizar así.

2 El coste de un algoritmo y el modelo de computación

Todo algoritmo, al ejecutarse en un ordenador real, consume dos **recursos fundamentales**:

- **Tiempo de ejecución:** medido en ciclos de procesamiento de la unidad central de procesamiento (CPU).
- **Espacio de memoria:** medido en la cantidad de memoria principal (RAM) adicional requerida para almacenar variables, tablas y estructuras auxiliares.

Un algoritmo consume dos recursos al ejecutarse: tiempo (ciclos de CPU) y espacio (memoria adicional que necesita).

2.1 ¿Por qué no medimos el coste en segundos?

Una tentación habitual en los primeros pasos computacionales es medir el tiempo que tarda un programa usando un cronómetro (con la orden `time`). Aunque la medición del tiempo de reloj (*wall-clock time*) es útil para comprobar el rendimiento práctico, **no sirve como medida teórica del algoritmo en sí**.

Si ejecutamos el mismo programa en un portátil antiguo, en un servidor de cómputo moderno, en una máquina con poca memoria o con el sistema operativo ocupado en otras tareas, el tiempo en segundos variará enormemente. Del mismo modo, el tiempo dependerá de si el código fue escrito en un lenguaje compilado de bajo nivel como C o en un lenguaje interpretado.

Para obtener una medida intrínseca y universal del algoritmo, la informática teórica utiliza el **conteo de operaciones elementales** en función del tamaño de los datos de entrada, habitualmente denotado por la variable n .

El coste de un algoritmo se mide contando operaciones elementales en función del tamaño de la entrada (n) en vez de en segundos, porque el tiempo en segundos depende de la máquina, la carga del sistema y el lenguaje, no solo del algoritmo.

2.2 Definición del tamaño de entrada en problemas multivariantes

En los ejemplos introductorios solemos resumir el tamaño de la entrada en una única variable n (como el número de elementos de una lista). Sin embargo, en biología computacional la entrada es habitualmente **multivariable**:

- En análisis de secuencias: M lecturas de longitud L cada una (la entrada total tiene tamaño $M \times L$).
- En alineamiento por pares: dos secuencias de longitudes n y m (la matriz de programación dinámica tiene orden $O(n \cdot m)$).
- En redes y vías metabólicas: un grafo con $|V|$ nodos (genes/metabolitos) y $|E|$ interacciones (aristas).

Antes de calcular cualquier complejidad, la primera pregunta obligatoria del científico es: *¿qué parámetros describen cuantitativamente el tamaño de los datos de entrada?*

2.3 El modelo RAM de operaciones elementales

Esta cuantificación se apoya en el modelo clásico de la máquina de acceso aleatorio (*Random Access Machine* o modelo RAM):

- La **memoria** se modela como un vector continuo de celdas con dirección propia donde leer o escribir datos toma tiempo constante ($O(1)$).
- La **CPU** ejecuta instrucciones elementales (sumas, restas, multiplicaciones, comparaciones lógicas, asignaciones y saltos de control) sobre palabras de tamaño fijo. Cada una de estas operaciones elementales se contabiliza con un coste unitario constante.

En el modelo RAM, la memoria es una sucesión de direcciones y la CPU ejecuta operaciones aritméticas, lógicas, de entrada/salida y de control sobre palabras de tamaño fijo; cada una de esas operaciones se cuenta como de coste constante.

2.4 Conteo de operaciones en tres algoritmos de ejemplo

Para comprender de forma práctica cómo se cuentan las operaciones elementales, analicemos tres algoritmos en Bash que resuelven exactamente la misma tarea: mostrar los N primeros múltiplos de 3. Los tres dan la misma salida. Lo que los diferencia es cuánto trabajo cuesta producirla.

```
# Algoritmo A: un bucle simple directo
for ((i = 1; i <= N; i++)); do
    echo $((i * 3))
done

# Algoritmo B: bucles anidados con sumas acumuladas
for ((i = 1; i <= N; i++)); do
    sum=0
    for ((j = 1; j <= i; j++)); do
        sum=$((sum + 3))
    done
    echo "$sum"
done

# Algoritmo C: un solo bucle con acumulador
sum=0
for ((i = 1; i <= N; i++)); do
    sum=$((sum + 3))
    echo "$sum"
done
```

Desglose del Algoritmo A (Lineal):

1. Inicialización: $i=1$ (1 operación).
2. En cada una de las N iteraciones:
 - Comparación de control: $i \leq N$ (1 op).
 - Multiplicación aritmética: $i * 3$ (1 op).
 - Emisión por salida estándar: `echo` (1 op).
 - Incremento y asignación: $i++$ (2 ops).

Total por vuelta: 5 operaciones.

El número total de operaciones elementales es $T_A(N) = 5N + 1$: cinco por vuelta y una de inicialización. Para $N = 10$ son 51, que es exactamente lo que cuenta el oráculo. El trabajo crece de forma **lineal** con N : al multiplicar N por 10, el tiempo se multiplica por 10.

El algoritmo A hace una operación de comparación, una de multiplicación y una de impresión por cada una de las N vueltas del bucle, más el establecimiento inicial de la variable: $5N + 1$ operaciones en total, que es $O(N)$.

Se reproduce con `verification/chapter_results.sh classify_single 10`.

Desglose del Algoritmo B (Cuadrático):

En el Algoritmo B, el bucle interior se ejecuta 1 vez en la primera iteración, 2 veces en la segunda, 3 en la tercera, y así sucesivamente hasta N veces en la última.

El número total de ejecuciones del cuerpo interior viene dado por la célebre suma de Gauss:

$$\sum_{i=1}^N i = 1 + 2 + 3 + \dots + N = \frac{N(N+1)}{2} = \frac{1}{2}N^2 + \frac{1}{2}N$$

El coste total de este algoritmo incluye un término en N^2 . A diferencia del Algoritmo A, si multiplicamos N por 10 (de $N = 100$ a $N = 1000$), el tiempo de ejecución se multiplica aproximadamente por 100.

El algoritmo B ejecuta su bucle interior i veces en cada una de las N vueltas del bucle exterior: el total de vueltas del bucle interior es $1 + 2 + \dots + N = N(N+1)/2$, del orden de N^2 , $O(N^2)$.

Desglose del Algoritmo C (lineal, con otra constante).

Dos operaciones de inicialización, $\text{sum}=0$ e $i=1$, y seis por vuelta: comparación, suma, asignación de sum , emisión, incremento y asignación de i . En total $T_C(N) = 6N + 2$.

Concepto. Aquí está el punto del apartado. Los algoritmos A y C hacen trabajos distintos, cuentan operaciones distintas y tienen constantes distintas, $5N + 1$ frente a $6N + 2$, pero los dos pertenecen a $O(N)$: al duplicar N , los dos duplican su trabajo. El algoritmo B, en cambio, cambia de clase. Eso es exactamente lo que la notación asintótica conserva y lo que descarta. Distingue A y C de B, y no distingue A de C. Cuando alguien dice que dos algoritmos «son igual de eficientes» porque comparten notación, está diciendo algo más débil de lo que parece: para N pequeño, la constante decide, y la constante es justamente lo que la notación tira.

Comprueba. Prediga cuántas vueltas dará el bucle interior del Algoritmo B con $N = 10$ antes de ejecutarlo. La suma de Gauss da $\frac{10 \times 11}{2} = 55$. Compruébelo, y compruebe también los otros dos: con el mismo N , el Algoritmo A hace 51 operaciones y el C hace 62. Los tres números son del mismo orden de magnitud, y sin embargo uno de los tres crece de forma distinta. ¿Cuál, y cómo lo sabe sin volver a medir?

3 Notación Asintótica: el lenguaje de la eficiencia

ESENCIAL

Al analizar algoritmos, los detalles menores de implementación (como si un bucle hace $5N + 2$ o $3N + 7$ operaciones) carecen de importancia cuando los datos crecen hacia millones de elementos. Lo que determina si un pipeline bioinformático es viable o inviable es su **tasa de crecimiento asintótico**.

La **notación Big-O** (O) es el estándar matemático universal para describir cómo escala el tiempo o la memoria de un algoritmo cuando el tamaño de entrada $n \rightarrow \infty$.

3.1 Las tres reglas de simplificación de la notación Big-O

1. **Conservar el término dominante:** si una función de coste tiene múltiples términos ($f(n) = 3n^2 + 500n + 10\,000$), para valores grandes de n el término cuadrático n^2 eclipsa totalmente a los demás.
2. **Descartar constantes multiplicativas:** $5n$ y $100n$ pertenecen ambos a la clase $O(n)$, ya que las constantes dependen de la velocidad de la CPU o del compilador, no del comportamiento intrínseco del algoritmo.
3. **Describir el comportamiento asintótico:** evaluar el límite cuando n crece sin cota.

La notación O sigue tres reglas: conservar el término dominante, descartar las constantes multiplicativas y describir el comportamiento asintótico (la tendencia cuando n crece sin límite).

3.2 Escala de complejidades comunes

Notación	Nombre	Efecto al duplicar n	Ejemplo biológico
$O(1)$	Constante	No cambia	Acceso a base por índice
$O(\log n)$	Logarítmica	Suma una operación constante	Búsqueda binaria en tabla
$O(n)$	Lineal	Se duplica ($2\times$)	Conteo de GC en secuencia
$O(n \log n)$	Cuasilineal	Algo más del doble ($2.2\times$)	Ordenación con Mergesort
$O(n^2)$	Cuadrática	Se cuadruplica ($4\times$)	Comparación de pares (todas con todas)
$O(2^n)$	Exponencial	Se eleva al cuadrado	Ensamblaje ingenuo sin podar
$O(n!)$	Factorial	Explosión combinatoria	Alineamiento múltiple por fuerza bruta

Con $n = 1\,000\,000$, un algoritmo $O(n)$ hace un millón de pasos, uno $O(n^2)$ hace un billón, y uno $O(2^n)$ es directamente irrealizable.

El punto de cruce: constante oculta frente a tasa de crecimiento.

Un error conceptual habitual es suponer que un algoritmo $O(n)$ es siempre más rápido que uno $O(n^2)$ para cualquier tamaño de entrada n . La notación asintótica describe la tendencia cuando $n \rightarrow \infty$, no el tiempo absoluto sobre entradas pequeñas.

Consideremos dos algoritmos con tiempos de trabajo abstractos $T_A(n) = 50n$ (lineal con constante alta) y $T_B(n) = n^2$ (cuadrático con constante baja):

- Para $n = 20$: $T_A(20) = 50 \times 20 = 1000$, mientras que $T_B(20) = 20^2 = 400$. Para entradas pequeñas, el algoritmo cuadrático realiza menos trabajo.
- Para $n = 50$: $T_A(50) = 2500$ y $T_B(50) = 2500$ (punto de cruce).
- Para $n = 1000$: $T_A(1000) = 50\,000$, mientras que $T_B(1000) = 1\,000\,000$. A partir del punto de cruce, la tasa cuadrática domina y su desventaja crece sin límite.

Por esta razón, las librerías científicas optimizadas combinan con frecuencia algoritmos cuadráticos sencillos para casos diminutos y conmutan a algoritmos asintóticamente superiores cuando los datos crecen (estrategia híbrida que estudiaremos en TC-CH06).

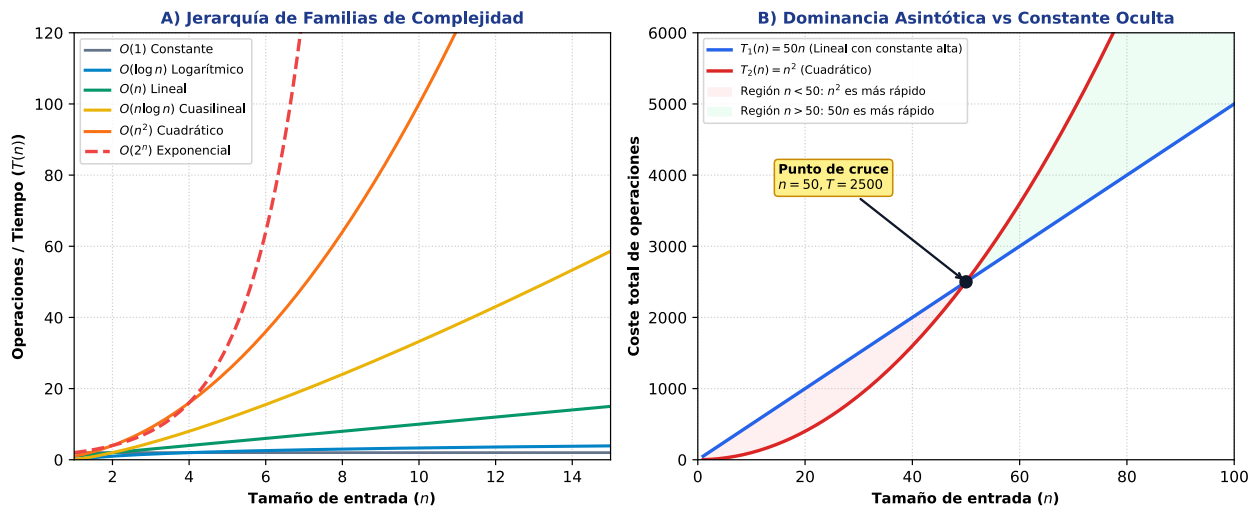


Figura 1: A) Jerarquía de crecimiento de las principales familias asintóticas. B) Detalle analítico del punto de cruce en $n = 50$ entre $T_1(n) = 50n$ y $T_2(n) = n^2$, ilustrando por qué la constante oculta domina para entradas pequeñas ($n < 50$) pero el orden de complejidad domina para problemas a escala genómica ($n \gg 50$). Figura original adaptada de las presentaciones docentes.

Comprueba. Ordene de menor a mayor coste, para n grande, estas ocho funciones: n , $n/2$, $\log n$, $n \log n$, n^2 , 2^n , n^3 y $n!$. Dos de ellas pertenecen a la misma clase asintótica. ¿Cuáles, y por qué? Si duda, calcule el cociente entre las dos cuando n tiende a infinito: si sale una constante, están en la misma clase.

3.3 Cotas superior (O), inferior (Ω) y ajustada (Θ)

En el análisis de algoritmos formal se distinguen tres cotas asintóticas fundamentales:

- **Cota superior $O(g(n))$ (Big-O):** garantiza que el algoritmo **no tardará más** que un múltiplo constante de $g(n)$ para entradas suficientemente grandes. Es la cota más utilizada en la práctica porque ofrece una garantía de peor caso.
- **Cota inferior $\Omega(g(n))$ (Big-Omega):** garantiza que el algoritmo tardará **al menos** un múltiplo de $g(n)$.
- **Cota ajustada $\Theta(g(n))$ (Big-Theta):** indica que el algoritmo tarda **exactamente** tanto como $g(n)$, estando acotado superior e inferiormente por la misma clase asintótica.

Cota asintótica no es sinónimo de caso analizado.

La notación $O(g(n))$ expresa una cota superior matemática sobre la función que se esté evaluando. Esa función puede ser el coste en el peor caso, en el caso promedio o en el mejor caso, según se declare explícitamente.

En ingeniería computacional se analiza prioritariamente el peor caso porque proporciona una garantía sobre el tiempo máximo, pero la notación Big-O y el «peor caso» son conceptos independientes: primero se fija el escenario o caso a estudiar (Sección 3.4) y posteriormente se acota asintóticamente su función de coste con O , Ω o Θ .

$O(g(n))$ representa una cota superior asintótica (la función no crece más rápido que $g(n)$ salvo constante); $\Omega(g(n))$ representa una cota inferior (crece al menos como $g(n)$); y $\Theta(g(n))$ representa una cota ajustada exacta. La notación de cota es independiente de si se analiza el peor caso, el caso promedio o el mejor caso.

3.4 Peor caso, mejor caso y caso promedio

El rendimiento de un algoritmo puede variar no solo según el tamaño n de la entrada, sino según la **disposición de los datos**:

- **Mejor caso:** la configuración más favorable de los datos (por ejemplo, buscar un elemento y que esté en la primera posición: $O(1)$).
- **Peor caso:** la configuración más desfavorable (el elemento buscado está al final o no existe: $O(n)$). Es la métrica estándar de garantía de ingeniería.
- **Caso promedio:** el comportamiento esperado sobre entradas aleatorias o realistas.

Se distinguen peor caso, caso medio y mejor caso de un algoritmo; el análisis de coste se aplica tanto al tiempo como al espacio adicional que necesita.

3.5 Clasificación de patrones elementales

Un bucle simple sobre una entrada de tamaño n es $O(n)$; un bucle anidado sobre la misma entrada es $O(n^2)$; un bucle que reduce n a la mitad en cada paso hasta llegar a 1 hace del orden de $\log_2 n$ pasos, $O(\log n)$.

Se reproduce con `verification/chapter_results.sh classify_single 10`,
`classify_nested` y `classify_halving`.

3.6 Demostración matemática del término dominante

ESTUDIO

Una duda frecuente al aprender notación asintótica es qué significa exactamente que el término $3n^2$ sea «dominante» en la expresión $f(n) = 3n^2 + 500n + 200$.

Analicemos la proporción que representa el término cuadrático $3n^2$ sobre el total:

- Para $n = 10$: $3(10^2) = 300$, frente a un total de $300 + 5000 + 200 = 5500$ (representa solo el 5.4%).
- Para $n = 1000$: $3(1000^2) = 3\,000\,000$, frente a un total de $3\,500\,200$ (representa el 85.7%).
- Para $n = 100\,000$: $3(100\,000^2) = 30\,000\,000\,000$, frente a $30\,050\,000\,200$ (representa el 99.83%).

A medida que n crece, el término de mayor exponente explica casi el 100% del coste computacional. Por ello, la regla formal descarta los términos de orden inferior ($500n + 200$) y posteriormente la constante multiplicativa (3), quedando formalmente como $O(n^2)$.

Para n suficientemente grande, el término dominante completo, $3n^2$ (con su constante), representa la práctica totalidad del valor de $3n^2 + 500n + 200$ —85,7% en $n = 1000$, 99,8% en $n = 100\,000$ —; la notación O correcta es $O(n^2)$, no « $O(3n^2 + 500n)$ », porque una vez identificado el término dominante completo se descartan tanto los términos de orden inferior como su constante multiplicativa.

Se reproduce con `verification/chapter_results.sh dominant_term_share 1000`.

4 Las cinco grandes familias de diseño algorítmico

ESENCIAL

Ante un problema computacional nuevo en bioinformática, los científicos no reinventan la rueda desde cero: aplican uno de los cinco grandes **paradigmas de diseño algorítmico**.

4.1 1. Fuerza bruta y búsqueda exhaustiva

La estrategia de **fuerza bruta** genera y comprueba sistemáticamente todas las soluciones posibles del espacio de búsqueda hasta encontrar la correcta o la óptima. Garantiza siempre encontrar la respuesta correcta si existe, pero su coste suele escalar de forma exponencial ($O(2^n)$) o factorial ($O(n!)$), volviéndose impracticable para entradas reales.

La fuerza bruta explora todas las soluciones posibles de un problema y garantiza encontrar la óptima, a costa de un coste típicamente $O(2^n)$ o $O(n!)$ que la hace inviable salvo para entradas pequeñas.

Caso resuelto: Búsqueda de sitios de corte de restricción.

Consideremos la tarea de localizar todas las dianas de corte de la enzima de restricción *EcoRI* (motivo GAATTC, de longitud $k = 6$) en una secuencia de ADN de longitud n . La fuerza bruta compara el motivo contra todas las $n - k + 1$ posiciones posibles de la secuencia.

```
funcion mapaDeRestriccion(secuencia, patron):
  posiciones <- []
  n <- longitud(secuencia)
  k <- longitud(patron)
  para i desde 0 hasta n-k:          # las n-k+1 posiciones posibles
    si secuencia[i .. i+k-1] == patron:
      agregar i a posiciones
  devolver posiciones                # ninguna posicion se descarta a priori
```

Buscar por fuerza bruta las apariciones de una subcadena de longitud k en una secuencia de longitud n compara la subcadena contra las $n - k + 1$ posiciones posibles: $O(n \cdot k)$ en el peor caso, una por comparación de hasta k caracteres en cada posición.

Se reproduce con `verification/chapter_results.sh restriction_sites GAATTCGAATTC GAATTC`.

4.2 2. Algoritmos voraces (*Greedy*)

Un algoritmo **voraz** construye la solución paso a paso, tomando en cada etapa la decisión que parece **óptima localmente**, con la esperanza de alcanzar el óptimo global. Los algoritmos voraces son extraordinariamente rápidos (típicamente $O(n)$ o $O(n \log n)$), pero tienen una limitación crítica: **no siempre garantizan el óptimo global**.

Un algoritmo voraz toma en cada paso la decisión que parece mejor localmente; solo es correcto si se demuestra que las decisiones locales óptimas construyen la solución global óptima. «Parece óptimo» no es una demostración.

```
funcion cambioVoraz(objetivo, monedas_descendente):
  resultado <- []
  restante <- objetivo
  para cada moneda en monedas_descendente:  # de mayor a menor valor
    mientras moneda <= restante:
      agregar moneda a resultado
      restante <- restante - moneda        # decision local, nunca se revisa
  devolver resultado
```

El contraejemplo del cambio de monedas.

Supongamos que debemos devolver un cambio de **6 unidades** disponiendo de monedas con valores $\{1, 3, 4\}$.

- **Estrategia voraz:** Toma la moneda más grande posible (4), quedando 2 por cubrir. No cabe otra de 4 ni de 3, por lo que toma una de 1 (queda 1) y otra de 1 (queda 0). Resultado: **3 monedas** (4 + 1 + 1).
- **Solución óptima real:** Tomar dos monedas de 3. Resultado: **2 monedas** (3 + 3).

El algoritmo voraz falló porque la elección local «tomar la moneda de 4» bloqueó la combinación global óptima (3 + 3).

Con monedas de valor {1,3,4} y un cambio de 6, el método voraz (tomar siempre la moneda más grande que quepa) usa 3 monedas (4 + 1 + 1), mientras que la solución óptima usa 2 (3 + 3): el voraz no alcanza el óptimo en este caso.

Se reproduce con `verification/chapter_results.sh greedy_change_1_3_4 6`.

```

ALGORITMO: Seleccion_Voraz_Fragmentos_Restriccion
ENTRADA:
- F: Lista de n fragmentos genomicos, donde cada fragmento i tiene (inicio_i, fin_i) con inicio_i < fin_i
SALIDA:
- S: Subconjunto disjunto maximo de fragmentos no solapantes
COMPLEJIDAD: Tiempo O(n log n) por la ordenacion inicial, Espacio auxiliar O(n)

1. INICIALIZACION:
# Criterio voraz optimo: ordenar los fragmentos por tiempo/posicion de finalizacion ascendente
F_ordenado <- Ordenar(F, clave=fin ascendente)
S <- Conjunto_Vacio()
ultimo_fin <- -INFINITO

2. BUCLE VORAZ:
Para cada fragmento f = (inicio, fin) en F_ordenado:
  Si f.inicio >= ultimo_fin entonces:
    # El fragmento es compatible con los seleccionados previamente
    Agregar f a S
    ultimo_fin <- f.fin # Decision irrevocable localmente optima

3. TERMINACION Y RETORNO:
Retornar S (Se demuestra que |S| alcanza el maximo teorico global)

```

Problema general frente a sistema de monedas.

Con las denominaciones del euro (1, 2, 5, 10, 20 y 50 céntimos, 1 y 2 euros) el método voraz produce siempre el óptimo, porque ese sistema es canónico. Devolver 77 céntimos, por ejemplo, sale bien tomando siempre la moneda más grande que quepa: 50, 20, 5 y 2, cuatro monedas, y no hay forma de hacerlo con menos. Pruebe usted mismo unos cuantos importes; no encontrará ninguno que falle.

Y sin embargo el mismo método falla con las denominaciones {1,3,4}, como veremos enseguida. La conclusión que importa es precisa: la corrección de un algoritmo voraz depende del **sistema de denominaciones** con el que trabaja. Es una propiedad del conjunto de monedas, no del importe concreto que haya que devolver. Por eso no basta con probar el algoritmo sobre unos cuantos casos: hay que demostrar que el sistema tiene la propiedad, o buscar el contraejemplo.

Un algoritmo se define para un problema general; una instancia es el caso concreto sobre el que opera (por ejemplo, qué denominaciones de moneda hay disponibles). El mismo algoritmo voraz de cambio de moneda es óptimo para las monedas del sistema euro pero no para {1,3,4}: la corrección de un voraz es una propiedad del sistema de denominaciones, no del importe concreto.

4.3 3. Divide y vencerás y recursión

El paradigma de **divide y vencerás** descompone un problema en subproblemas más pequeños del mismo tipo, resuelve los subproblemas de forma independiente mediante llamadas recursivas y finalmente **combina** sus soluciones para resolver el problema original.

Una solución recursiva resuelve un problema reduciéndolo a versiones más pequeñas de sí mismo; divide y vencerás parte el problema en subproblemas independientes, los resuelve recursivamente y combina sus

resultados.

Anatomía de la recursión:

Toda función recursiva correcta debe poseer dos elementos indispensables:

1. **Caso base:** la condición de parada simple que devuelve un resultado directo sin realizar llamadas adicionales.
2. **Llamada recursiva:** la invocación a sí misma que reduce el tamaño del problema acercándose estrictamente al caso base.

```
fib(n):  
    si n <= 1: devolver n  
    devolver fib(n-1) + fib(n-2)
```

```
fib(4)  
+-- fib(3)  
|   +-- fib(2)  
|   |   +-- fib(1)  
|   |   |-- fib(0)  
|   |-- fib(1)  
|-- fib(2)      <- recalculado desde cero  
   +-- fib(1)  
   |-- fib(0)
```

Figura 2: Árbol de llamadas de `fib(4)`: `fib(2)` se calcula dos veces de forma independiente, cada vez repitiendo su propio subárbol completo. Figura original.

En el árbol de llamadas de `fib(5)` sin memoizar, `fib(2)` se calcula 3 veces de forma independiente; el número total de llamadas crece de forma exponencial, $O(2^n)$.

Se reproduce con `verification/chapter_results.sh fib_naive_calls 5`.

Peligro de desbordamiento de pila (*Stack Overflow*):

Cada llamada a una función consume memoria en la pila del sistema (*stack*) para almacenar variables locales y la dirección de retorno. Si falta el caso base, la recursión entrará en un bucle infinito que agotará la memoria de la pila, provocando el colapso inmediato del programa.

Toda solución recursiva requiere un caso base (la condición simple que detiene la recursión sin más llamadas) y una llamada recursiva que reduzca el problema hacia él; sin un caso base alcanzable, cada llamada consume memoria de pila adicional sin límite hasta agotarla (desbordamiento de pila).

La iteración usa menos memoria que la recursión porque no acumula llamadas en la pila; la recursión es más natural para problemas de estructura repetitiva o divisible, como recorrer árboles y grafos o aplicar divide y vencerás, a costa de esa memoria adicional por cada llamada abierta.

4.4 4. Programación dinámica

La **programación dinámica** es una de las técnicas más potentes de la computación científica y la base de los algoritmos de alineamiento de secuencias biológicas (como Needleman-Wunsch y Smith-Waterman). Se aplica cuando un problema cumple dos propiedades:

1. **Subestructura óptima:** la solución óptima del problema global se construye a partir de las soluciones óptimas de sus subproblemas.
2. **Subproblemas solapados:** los mismos subproblemas aparecen y se necesitan una y otra vez durante el cálculo.

La idea maestra es la **memoización**: calcular la solución de cada subproblema exactamente una sola vez y guardarla en una tabla o matriz para reutilizarla de inmediato en tiempo constante ($O(1)$).

La programación dinámica se aplica cuando un problema tiene subestructura óptima y subproblemas solapados; memoizar (guardar el resultado de cada subproblema la primera vez que se calcula) evita recalcularlo, convirtiendo una recursión exponencial en un algoritmo eficiente.

Memoizando fib, cada valor de 0 a n se calcula exactamente una vez: el número de cálculos pasa de exponencial a $O(n)$.

Se reproduce con `verification/chapter_results.sh fib_memo_calls 10`.

```
ALGORITMO: Fibonacci_Comparativa_Complejidad
ENTRADA:
- n: Entero no negativo representando la posicion en la serie de Fibonacci
SALIDA:
- F_n: El n-esimo numero de Fibonacci

# VARIANTE 1: INGENUA RECURSIVA (inviabile: tiempo  $O(2^n)$ , espacio de pila  $O(n)$ )
Funcion Fibonacci_Ingenuo(n):
  Si n <= 0 entonces retornar 0
  Si n = 1 entonces retornar 1
  Retornar Fibonacci_Ingenuo(n - 1) + Fibonacci_Ingenuo(n - 2)

# VARIANTE 2: PROGRAMACION DINAMICA MEMOIZADA (Optima: Tiempo  $O(n)$ , Espacio  $O(1)$ )
ALGORITMO: Fibonacci_DP_Iterativo_Optimo(n):
  Si n <= 0 entonces retornar 0
  Si n = 1 entonces retornar 1

  prev2 <- 0    # F_{i-2}
  prev1 <- 1    # F_{i-1}

  Para i desde 2 hasta n:
    actual <- prev1 + prev2  # F_i = F_{i-1} + F_{i-2}
    prev2 <- prev1
    prev1 <- actual

  Retornar actual  # Calculado en exactamente (n-1) sumas elementales
```

Resolución del cambio de monedas con programación dinámica:

Definiendo $dp[c]$ como el número mínimo de monedas para formar el importe c , con $dp[0] = 0$:

$$dp[c] = 1 + \min_{\substack{m \in \{1,3,4\} \\ m \leq c}} dp[c - m]$$

```
funcion monedasMinimasDP(objetivo, monedas):
  dp[0] <- 0
  para c desde 1 hasta objetivo:
    dp[c] <- infinito
    para cada m en monedas:
      si m <= c:
        dp[c] <- min(dp[c], dp[c - m] + 1)
  devolver dp[objetivo]
```

El cambio de monedas con $\{1, 3, 4\}$ tiene subestructura óptima y subproblemas solapados (por ejemplo, $dp[3]$ se necesita para calcular $dp[4]$ y $dp[6]$); se resuelve con la relación de recurrencia $dp[c] = 1 + \min_{m \leq c} dp[c - m]$ con $dp[0] = 0$.

Sobre monedas $\{1, 3, 4\}$ y objetivo 6, la tabla de programación dinámica da $dp[0..6] = 0, 1, 2, 1, 1, 2$: el importe 6 se resuelve con 2 monedas ($dp[6] = 2$, correspondiente a $3 + 3$), el óptimo real, no las 3 monedas ($4 + 1 + 1$) que dio el algoritmo voraz sobre la misma instancia. La programación dinámica considera todas las combinaciones posibles a través de la tabla, sin descartar ninguna opción localmente antes de tiempo.

Se reproduce con `verification/chapter_results.sh coin_change_dp 6`.

Memoizar solo aporta cuando existen subproblemas solapados; en un algoritmo cuyos subproblemas son independientes y no se repiten (como la ordenación por mezcla), memoizar no ahorra cómputo y solo añade sobrecarga de memoria.

4.5 5. Vuelta atrás (*Backtracking*)

El paradigma de **vuelta atrás** (*backtracking*) es una optimización de la fuerza bruta para problemas de búsqueda combinatoria. En lugar de generar todas las combinaciones posibles y filtrarlas al final, el algoritmo construye soluciones parciales candidatas y **abandona inmediatamente** (*poda*) una rama en cuanto comprueba que no puede satisfacer las restricciones del problema.

```
backtracking(solucion_parcial, datos_restantes):
    si solucion_parcial es completa:
        si cumple los criterios: guardarla
        devolver
    para cada opcion posible en datos_restantes:
        agregar opcion a solucion_parcial
        backtracking(solucion_parcial, datos_restantes sin opcion)
        quitar opcion de solucion_parcial # deshacer: el paso que no tiene la fuerza bruta
```

Backtracking explora el espacio de soluciones abandonando una rama en cuanto se comprueba que no puede llevar a una solución válida, a diferencia de la fuerza bruta, que completa cada combinación antes de descartarla; su implementación típica se apoya en una pila que guarda el camino parcial.

Caso resuelto: Generación de motivos de ADN sin pares AA.

Supongamos que deseamos generar todas las secuencias de longitud $L = 4$ sobre el alfabeto $\{A, T\}$ con la restricción biológica de que nunca aparezcan dos adeninas consecutivas (AA).

- **Fuerza bruta sin podar:** Exploraría un árbol completo de $2^0 + 2^1 + 2^2 + 2^3 + 2^4 = 31$ nodos para generar 16 secuencias y descartar las inválidas.
- **Backtracking con poda:** En cuanto añade una A y la siguiente candidata es otra A, aborta la rama inmediatamente. Solo visita **19 nodos** y devuelve directamente las **8 cadenas válidas**.

Para generar cadenas de longitud 4 sobre $\{A, T\}$ sin dos «A» consecutivas, backtracking visita 19 nodos del árbol de búsqueda, frente a los 31 nodos que tendría el árbol completo sin podar ($2^0 + 2^1 + \dots + 2^4$); encuentra exactamente 8 cadenas válidas.

Se reproduce con `verification/chapter_results.sh backtracking_count 4`.

La predicción de estructura secundaria de ARN por emparejamiento de bases y el alineamiento simultáneo de tres o más secuencias son dos problemas que se pueden plantear como backtracking con poda; ambos son desarrollos de capítulos posteriores y no se implementan aquí.

5 La operación peligrosa: la explosión combinatoria oculta

ESTUDIO

En bioinformática, una de las trampas más devastadoras ocurre cuando un algoritmo con complejidad exponencial $O(2^n)$ se programa y prueba con entradas de juguete ($n = 10$) durante la fase de desarrollo. El algoritmo parece funcionar de forma instantánea ($2^{10} = 1024$ operaciones). Sin embargo, al aplicarlo al conjunto de datos real de laboratorio con $n = 50$, el número de operaciones salta a $2^{50} \approx 1.12 \times 10^{15}$, provocando el bloqueo total del servidor, saturación de la memoria RAM y pérdida de tiempo de computación.

Protocolo preventivo en 3 pasos ante la escala de datos:

1. **Análisis asintótico previo:** deducir la cota Big-O formal del algoritmo antes de escribir la primera línea de código.
2. **Prueba de escalado con duplicación ($2\times$):** ejecutar el algoritmo con entradas de tamaño creciente ($n =$

10, 20, 30, 40) midiendo el tiempo de ejecución. Si al duplicar n el tiempo se multiplica por 4, es cuadrático ($O(n^2)$); si se eleva exponencialmente, es $O(2^n)$.

3. **Validación de límites y guardia de tamaño:** establecer una comprobación preventiva en el código que rechace o alerte ante tamaños de entrada que excedan la capacidad computacional disponible.

6 Actividad de depuración: localizar fallos en algoritmos ajenos

ESTUDIO

Diagnosticar errores en algoritmos diseñados por otros o por herramientas de IA es una habilidad crítica. Analice el siguiente fragmento con tres errores conceptuales:

```
funcion buscarRepetidos(lista, n):
  para i desde 0 hasta n:      # Fallo 1
    para j desde 0 hasta n:    # Fallo 2
      si lista[i] == lista[j]: # Fallo 3
        devolver VERDADERO
  devolver FALSO
```

Tarea. Localice los tres fallos, corrija cada uno y clasifíquelo antes de leer el diagnóstico: ¿es un error de corrección, que hace que la función devuelva una respuesta equivocada, o una ineficiencia, que hace que tarde más de lo necesario? La distinción no es académica: uno invalida el resultado y el otro solo cuesta tiempo.

Diagnóstico.

1. **Desbordamiento de índice** (corrección). El bucle llega hasta n en lugar de $n - 1$ y accede a una posición que no existe. Según el lenguaje, esto aborta el programa o lee memoria ajena, que es peor porque no avisa.
2. **Comparaciones repetidas** (eficiencia). Con j recorriendo todo el rango, cada par se compara dos veces. Empezando en $i + 1$ se hace la mitad del trabajo y el resultado es el mismo: el algoritmo sigue siendo $O(n^2)$, pero con la constante dividida por dos.
3. **Falta la condición $i \neq j$** (corrección, y el importante). Tal como está, la función compara cada elemento consigo mismo, encuentra que son iguales y devuelve verdadero para *cualquier* lista no vacía. No es que dé un resultado inexacto: es que no mira los datos. Un fallo así pasa todas las pruebas que solo usan listas con repetidos.

Concepto. Fíjese en el orden de gravedad. El tercero produce siempre la respuesta correcta cuando la respuesta correcta es «sí», y por eso una batería de pruebas mal elegida lo aprueba. El segundo no afecta al resultado en absoluto. Al auditar código ajeno, empiece preguntando qué devuelve en el caso en que la respuesta debería ser «no»; después, cuánto cuesta.

Comprueba. Una vez corregidos los tres, queda la pregunta de diseño: ordenar primero la lista, con un coste de $O(n \log n)$, deja los duplicados adyacentes y permite detectarlos en una sola pasada. ¿Compensa? Depende de si espera encontrar el repetido pronto: la versión cuadrática puede terminar en la segunda comparación, y la que ordena paga siempre el coste completo antes de empezar a mirar.

7 Síntesis y diagnóstico de problemas algorítmicos

REFERENCIA

Técnica	Idea central	¿Óptimo?	Coste típico	Ejemplo en biología
Fuerza bruta	Probar todas las opciones	Sí	$O(2^n)$, $O(n!)$	Búsqueda exhaustiva de diana

Técnica	Idea central	¿Óptimo?	Coste típico	Ejemplo en biología
Voraz (<i>Greedy</i>)	Elegir lo mejor local	No siempre	$O(n)$, $O(n \log n)$	Selección de experimentos
Divide y vencerás	Dividir y combinar	Depende	$O(n \log n)$	Mergesort en secuencias
Prog. Dinámica	Subproblemas solapados	Sí	$O(n^2)$, $O(n^3)$	Alineamiento de secuencias
Vuelta atrás	Podar ramas inviables	Sí	variable; en el peor caso, sin nada que podar, sigue siendo $O(2^n)$	Plegamiento de ARN

Las cinco técnicas de diseño se sintetizan por idea central, optimalidad garantizada, coste típico y un ejemplo biológico: fuerza bruta (probar todas las soluciones, siempre óptima, $O(2^n)/O(n!)$, búsqueda exhaustiva de subcadena); voraz (elegir lo mejor en cada paso, no siempre óptimo, $O(n)/O(n \log n)$, selección de actividades); divide y vencerás (partir y combinar, depende, coste variable, árbol filogenético); programación dinámica (subproblemas solapados más memoización, óptima, $O(n^2)/O(n^3)$, alineamiento de secuencias); backtracking (podar ramas inválidas pronto, óptima, coste variable pero mejor que fuerza bruta, ensamblaje combinatorio restringido).

7.1 Tabla de diagnóstico de síntomas computacionales

Síntoma observado	Causa probable	Comprobación y corrección
El programa se congela al aumentar N de 15 a 30	Algoritmo con complejidad exponencial $O(2^n)$	Comprobar si hay subproblemas solapados y aplicar memoización
Error <i>Stack overflow</i> / desbordamiento	Recursión sin caso base o llamada que no reduce N	Verificar que el caso base esté al principio y sea alcanzable
La solución voraz devuelve un resultado subóptimo	La instancia no cumple la propiedad voraz	Plantear la relación de recurrencia con Programación Dinámica
El tiempo se multiplica por 4 al duplicar N	Bucle anidado oculto ($O(n^2)$)	Ordenar previamente ($O(n \log n)$) o usar indexación hash ($O(1)$)

7.2 Tabla de síntesis operativa con preguntas de control

Necesidad	Paradigma recomendado	Pregunta de control
Probar todo el espacio de soluciones	Fuerza bruta	¿Es n suficientemente pequeño ($n \leq 15$)?
Solución rápida con elecciones locales	Algoritmo voraz (<i>Greedy</i>)	¿Está demostrado que la elección local lleva al óptimo global?
Subproblemas independientes	Divide y vencerás	¿Cómo se combinan las soluciones parciales?
Subproblemas repetidos y solapados	Programación dinámica	¿Tengo memoria suficiente para la tabla de memoización?
Espacio combinatorio con restricciones	Backtracking con poda	¿Puedo detectar y abortar ramas inválidas en etapas tempranas?

8 Glosario esencial

- **Algoritmo:** secuencia finita y bien definida de pasos para resolver un problema computacional.
- **Notación Big-O (O):** cota asintótica superior que describe el orden de crecimiento del coste de un algoritmo.
- **Modelo RAM:** abstracción de computador donde las operaciones elementales de CPU y memoria toman tiempo unitario constante.
- **Fuerza bruta:** exploración exhaustiva de todas las soluciones posibles.
- **Algoritmo voraz (*Greedy*):** método que selecciona la opción óptima local en cada paso.
- **Divide y vencerás:** técnica que divide un problema en partes independientes, las resuelve y combina los resultados.
- **Recursión:** mecanismo por el cual una función se invoca a sí misma sobre una instancia reducida.
- **Caso base:** condición de terminación en una función recursiva.
- **Programación dinámica:** técnica para problemas con subestructura óptima y subproblemas solapados mediante memoización.
- **Memoización:** almacenamiento de resultados intermedios en una tabla para evitar recálculos.
- **Backtracking:** exploración combinatoria con retroceso y poda temprana de ramas inválidas.

9 Conexión con el curso y lo que viene después

El modelo de coste de este capítulo se usa sin volver a enseñarse en los siguientes:

- En **TC-CH04** y **TC-CH05**, formalizaremos las estructuras de datos fundamentales (lineales, árboles, tablas hash y grafos biológicos) analizando el coste temporal y espacial de cada operación de acceso y mutación.
- En **TC-CH06**, aplicaremos y compararemos sistemáticamente algoritmos de búsqueda y ordenación (lineal, binaria, QuickSort, Mergesort, Heapsort y Radix Sort), comprobando empíricamente cotas y constantes.
- En **TC-CH07** y **TC-CH08**, aplicaremos teoría de la información y modelos probabilísticos (cadenas de Markov y HMMs con el algoritmo de Viterbi).
- En **TC-CH09**, la programación dinámica resolverá el alineamiento de secuencias con Needleman–Wunsch y Smith–Waterman, y la heurística de BLAST permitirá explorar bases de datos a escala genómica.
- En **TC-CH10**, extenderemos el aprendizaje computacional hacia redes neuronales y representaciones aprendidas.

La formalización de complejidad asintótica y paradigmas de este capítulo fundamenta el análisis de estructuras en TC-CH04/05, la búsqueda y ordenación en TC-CH06, los modelos probabilísticos en TC-CH08, el alineamiento en TC-CH09 y el aprendizaje profundo en TC-CH10.

10 Fuentes y límites de uso

Este capítulo se apoya en la presentación docente de la asignatura sobre análisis de algoritmos, escrita por Álvaro Serrano Navarro. Las definiciones de las tres cotas asintóticas y la distinción entre cota y caso siguen a Cormen y colaboradores [3]; los ejemplos de aplicación biológica, a Compeau y Pevzner [2]; y el conteo de operaciones sobre código de Shell, a Allesina y Wilmes [1].

Todas las cuentas que aparecen en el texto se reproducen con `verification/chapter_results.sh` tal como están impresas, con sus argumentos. Cuando una cuenta del texto y una del oráculo discrepen, la del oráculo es la que manda: es la que se ejecuta.

Bibliografía

- [1] Stefano Allesina and Madlen Wilmes. *Computing Skills for Biologists: A Toolbox*. Princeton University Press, 2019. Ejemplos contrastivos de algoritmos; no sostiene ninguna afirmación en solitario.
- [2] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach*. 3rd edition, 2018. Encuadre bioinformático de la programación dinámica como motor del alineamiento, citado hacia TC-CH09.

- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009. Referencia canónica para la prueba del límite de la ordenación por comparación y la clasificación de paradigmas de diseño; no se reproduce, solo se cita.